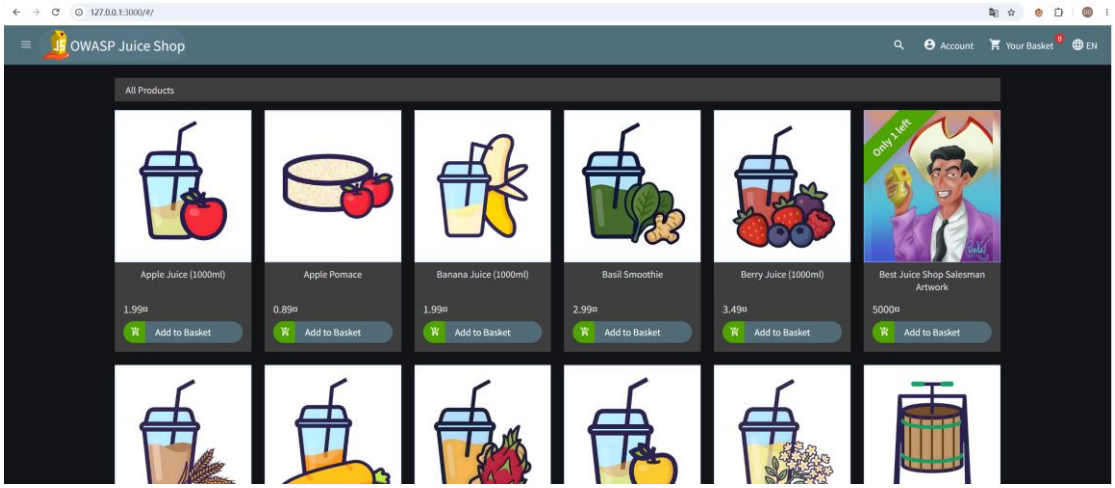
Lab 5: Automated Web Vulnerability Analysis Using Python

Environment setup



Part A — Understanding HTTP before writing a scanner

A1. Your first request

```
import requests

base_url = "http://localhost:3000"
response = requests.get(base_url, timeout=5)

print("HTTP status:", response.status_code)
print("Response length:", len(response.content), "bytes")
```

```
HTTP status: 200
Response length: 9984 bytes
```

What is the difference between response.status_code and len(response.content)?

response.status_code is the HTTP response status code indicating whether the request was successful (e.g., 200 for OK), while len(response.content) represents the size/length of the returned HTTP response body in bytes.

A2. Reading response headers and content

```
print("Content type:", response.headers.get("Content-Type"))
print("First 100 characters:", response.text[:100])
```

```
Content type: text/html; charset=UTF-8
First 100 characters: <!--
  ~ Copyright (c) 2014-2026 Bjoern Kimminich & the OWASP Juice Shop contributors.
  ~ SPDX-Lice
```

```
print("Content type:", response.headers.get("Content-Type"))
print("First 100 characters:", response.text[:50])
```

```
Content type: text/html; charset=UTF-8
First 100 characters: <!--
  ~ Copyright (c) 2014-2026 Bjoern Kimminich
```

Slicing [:50] extracts and returns only the first 50 characters of the response text (starting from index 0 up to 49), which shortens the output instead of printing the entire web page content.

A3. Sending a search parameter



A3. Sending a search parameter

The `params` argument builds the query string safely. The next request searches for the word `apple`. Notice that `requests` handles URL encoding for you.

```
search_url = base_url + "/rest/products/search"
r = requests.get(search_url, params={"q": "apple"}, timeout=5)

print("Requested URL:", r.url)
print("Status:", r.status_code)
print("Content type:", r.headers.get("Content-Type"))
print("First 150 characters:", r.text[:150])
```

```
Requested URL: http://localhost:3000/rest/products/search?q=apple
Status: 200
Content type: application/json; charset=utf-8
First 150 characters: {"status":"success","data":[{"id":1,"name":"Apple Juice (1000ml)","description":"The all-time classic.","price":1.99,"deluxePric
e":0.99,"image":"apple
```

Practice A: Run the example again using the search word `banana`. Which part of the request changed? Did the status code change? Do not assume that a different response length indicates a vulnerability.

```
search_url = base_url + "/rest/products/search"
r = requests.get(search_url, params={"q": "banana"}, timeout=5)

print("Requested URL:", r.url)
print("Status:", r.status_code)
print("Content type:", r.headers.get("Content-Type"))
print("First 150 characters:", r.text[:150])
```

```
Requested URL: http://localhost:3000/rest/products/search?q=banana
Status: 200
Content type: application/json; charset=utf-8
First 150 characters: {"status":"success","data":[{"id":6,"name":"Banana Juice (1000ml)","description":"Monkeys love it the most.","price":1.99,"deluxe
Price":1.99,"image":"
```

Practice A: Run the example again using the search word banana. Which part of the request changed? Did the status code change? Do not assume that a different response length indicates a vulnerability.

It changed the requested URL to http://localhost:3000/rest/products/search?q=banana. The HTTP status code remained unchanged at 200, and the response correctly updated to display the search results for banana juice.

A4. GET versus POST

A4. GET versus POST

GET typically retrieves information; POST typically submits data. The HTTP method alone does not determine whether a request is secure. In this demonstration, POST is sent to the search path **only to observe how the server handles an unsupported method**. A 404 or 405 is an ordinary outcome.

```
post_response = requests.post(
    search_url,
    data={"q": "apple"},
    timeout=5
)
print("POST status:", post_response.status_code)
print("POST response length:", len(post_response.content))
```

```
POST status: 500
POST response length: 2442
```

Practice B: In one or two sentences, compare the GET and POST results. Why does a `405` response not necessarily indicate a vulnerability?

Practice B: In one or two sentences, compare the GET and POST results. Why does a 405 response not necessarily indicate a vulnerability?

The GET request returned a status code of 200 with search results, while the POST request failed with a status code of 500 (or 405) because the endpoint does not support POST requests. A 405 response simply indicates that the server correctly blocks unsupported HTTP methods as expected, which is normal behavior and not a security vulnerability.

Part B — Python building blocks for a scanner

B1. Lists and loops

B1. Lists and loops ¶

A list stores several items. A `for` loop visits them one by one. This is how a scanner can test multiple inputs without duplicating code.

```
search_terms = ["apple", "banana", "orange", "lemon"]

for term in search_terms:
    print("Search term:", term)
```

```
Search term: apple
Search term: banana
Search term: orange
Search term: lemon
```

Practice C: Add `lemon` to the list and run the cell again. How many times does the loop run?

Practice C: Add lemon to the list and run the cell again. How many times does the loop run?

4.

B2. Dictionaries: one record per request

```
observation = {
    "method": "GET",
    "endpoint": "/rest/products/search",
    "status_code": r.status_code,
    "response_length": len(r.content),
    "content_type": r.headers.get("Content-Type")
}

print(observation)
print("Status:", observation["status_code"])
```

```
{'method': 'GET', 'endpoint': '/rest/products/search', 'status_code': 200, 'response_length': 504, 'content_type': 'application/json; charset=utf-8'}
Status: 200
```

Practice D: Add a `content_type` key to `observation` using `r.headers.get("Content-Type")`. Print that value.

Practice D: Add a content_type key to observation using r.headers.get("Content-Type"). Print that value.

B3. Combining a loop and a list of records

B3. Combining a loop and a list of records

`append()` adds a new dictionary to a list. Notice that this example uses only two search terms and records just two fields; **it is not the complete scanner**.

```
records = []

for term in ["apple", "banana"]:
    result = requests.get(search_url, params={"q": term}, timeout=5)
    records.append({"input": term, "status_code": result.status_code, "response_length": len(r.content)})

print(records)
```

```
[{'input': 'apple', 'status_code': 200, 'response_length': 504}, {'input': 'banana', 'status_code': 200, 'response_length': 504}]
```

Practice E: Add `response_length` to each dictionary. Explain why a list of dictionaries is helpful for generating a report.

Practice E: Add response_length to each dictionary. Explain why a list of dictionaries is helpful for generating a report.

A list of dictionaries is helpful for generating a report because it stores structured data for each request (such as input, status code, and response length) in key-value pairs. This organized format makes it easy to iterate through, filter, analyze, and format the collected scan results into a clean final report.

B4. Writing a simple function

B4. Writing a simple function

A function gives a repeated task a name. Parameters are inputs to the function, and `return` sends a value back. This example returns only the HTTP status code, leaving the full scanner design to you.

```
def get_status(search_word):
    result = requests.get(search_url, params={"q": search_word}, timeout=5)
    return result.status_code

print(get_status("apple"))
```

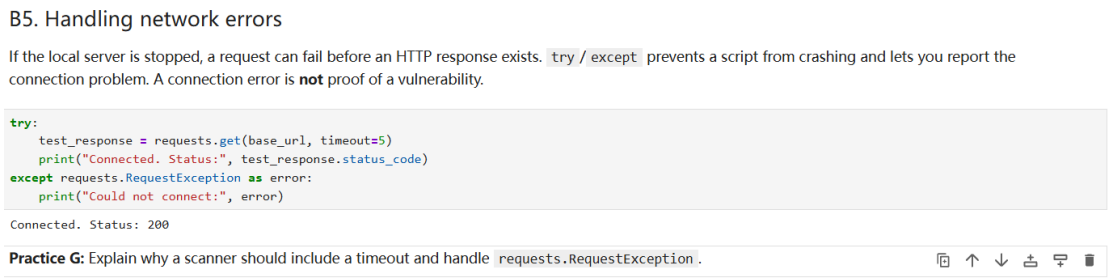
```
200
```

Practice F: Call `get_status("banana")`. Then explain what `search_word` means inside the function. You will decide later whether your scanner needs a function that returns a whole dictionary instead of a single number.

Practice F: Call get_status("banana"). Then explain what search_word means inside the function. You will decide later whether your scanner needs a function that returns a whole dictionary instead of a single number.

In the function, search_word acts as a parameter that receives the input value (such as "banana") and passes it into the query parameters to fetch the HTTP status code. Calling get_status("banana") will execute the request for "banana" and return 200.

B5. Handling network errors

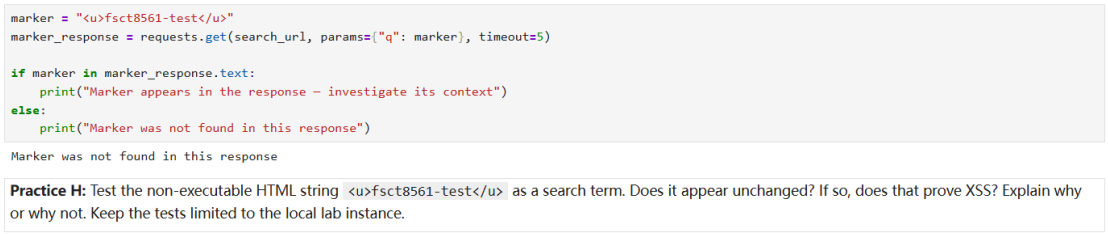


Practice G: Explain why a scanner should include a timeout and handle requests.RequestException

A scanner should include a timeout to prevent it from hanging indefinitely if a target server is unresponsive, and it should handle requests.RequestException to catch network errors and prevent the script from crashing, allowing the scan to continue smoothly.

Part C — Recognizing potential web security indicators

C1. Reflected input and XSS



Practice H: Test the non-executable HTML string <u>fsct8561-test</u> as a search term. Does it appear unchanged? If so, does that prove XSS? Explain why or why not. Keep the tests limited to the local lab instance.

The string did not appear unchanged. Even if it had, reflection alone does not establish XSS because the browser might escape the text, return it as JSON, or never execute it, meaning manual verification is still required.

C2. SQL injection indicators

```
normal_input = "apple"
probe_input = "'"

normal = requests.get(search_url, params={"q": normal_input}, timeout=5)
probe = requests.get(search_url, params={"q": probe_input}, timeout=5)

print("Normal input status:", normal.status_code)
print("Probe input status:", probe.status_code)
```

```
Normal input status: 200
Probe input status: 200
```

Practice I: Explain what you observed. Why is comparing the probe response to a normal baseline more informative than looking at the probe alone?

Practice I: Explain what you observed. Why is comparing the probe response to a normal baseline more informative than looking at the probe alone?

Comparing the probe response to a normal baseline helps identify unexpected changes (like status code differences, error messages, or response length shifts) caused specifically by the syntax probe, whereas looking at the probe alone lacks a reference point to determine if the behavior is truly anomalous or just standard application behavior.

C3. Checking for database error messages

```
error_phrases = ["SQLITE_ERROR", "SQL syntax", "Unclosed quotation mark"]
for error_phrase in error_phrases:
    if error_phrase.lower() in probe.text.lower():
        print("Database-related error text found; manual review required")
    else:
        print("No matching database error text in this response")
```

```
No matching database error text in this response
No matching database error text in this response
No matching database error text in this response
```

Practice J: Suggest two additional database-error phrases that a detector might check. Explain why a matching phrase alone cannot establish successful SQL injection.

Practice J: Suggest two additional database-error phrases that a detector might check. Explain why a matching phrase alone cannot establish successful SQL injection.

A matching error phrase alone cannot establish successful SQL injection because the message might just be a generic error page, standard database feedback, or safely handled output rather than actual unauthorized data access or code execution, meaning manual verification is still required.

C4. Security response headers

```
headers_to_check = ["Content-Security-Policy", "X-Content-Type-Options"]

for name in headers_to_check:
    if name in response.headers:
        print(name, "is present")
    else:
        print(name, "is not present; investigate its relevance")

print(response.headers.get("Server"))
```

```
Content-Security-Policy is not present; investigate its relevance
X-Content-Type-Options is present
None
```

Practice K: Look at the `Server` response header using `.get("Server")`. What information, if any, is revealed? Why is the presence of a `Server` header not automatically a critical vulnerability?

Practice K: Look at the Server response header using .get("Server"). What information, if any, is revealed? Why is the presence of a Server header not automatically a critical vulnerability?

The Server response header typically reveals the software name and version of the underlying web server (such as Express, Apache, or Nginx). However, its presence is not automatically a critical vulnerability because it only serves as an informational indicator of the technology stack rather than an active, exploitable security flaw or unauthorized access vector.

C5. Conditional alerts and avoiding duplicates

```
alerts = []
seen = set()

for message in ["Missing CSP", "Missing CSP", "Missing X-Content-Type-Options"]:
    if message not in seen:
        seen.add(message)
        alerts.append(message)

print(alerts)
```

```
['Missing CSP', 'Missing X-Content-Type-Options']
```

Practice L: Why does the list contain only two alerts even though the loop processes three messages? In your final detector, decide what makes two alerts the "same" finding (for example, category + endpoint + observation).

Practice L: Why does the list contain only two alerts even though the loop processes three messages? In your final detector, decide what makes two alerts the "same" finding (for example, category + endpoint + observation).

The list contains only two alerts because the set (seen) tracks and filters out duplicate messages, ensuring that identical warnings (such as the second "Missing CSP") are only recorded once. In your final detector, determining what makes two alerts the "same" finding typically involves combining multiple unique attributes, such as category, endpoint, and observation.

C6. Writing observations to a CSV file

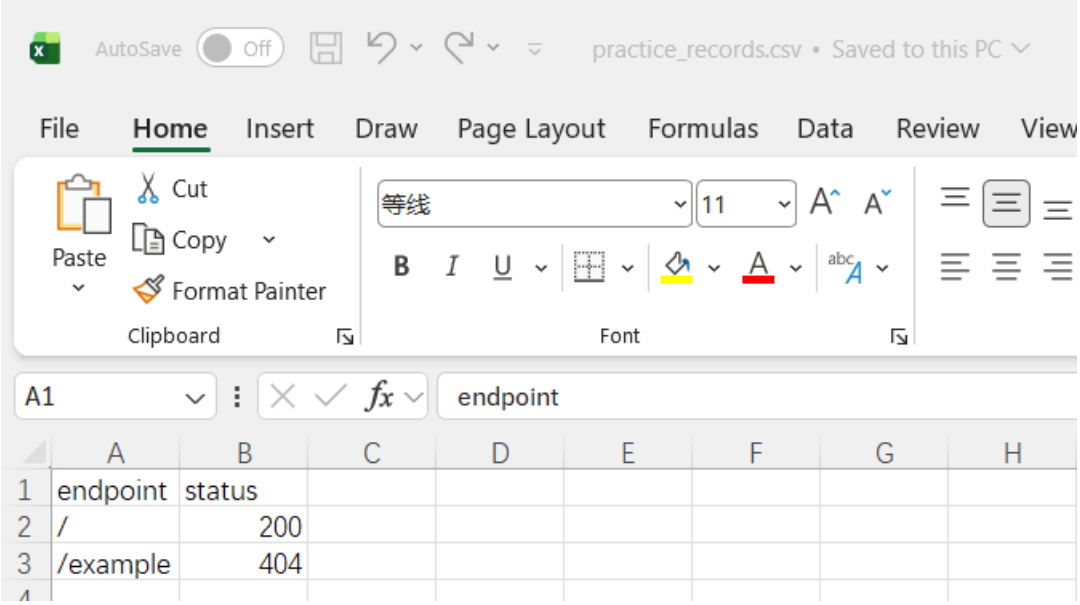
```
import csv

practice_records = [
    {"endpoint": "/", "status": 200},
    {"endpoint": "/example", "status": 404}
]

with open("practice_records.csv", "w", newline="", encoding="utf-8") as file:
    writer = csv.DictWriter(file, fieldnames=["endpoint", "status"])
    writer.writeheader()
    writer.writerows(practice_records)

print("Saved practice_records.csv")
```

```
Saved practice_records.csv
```



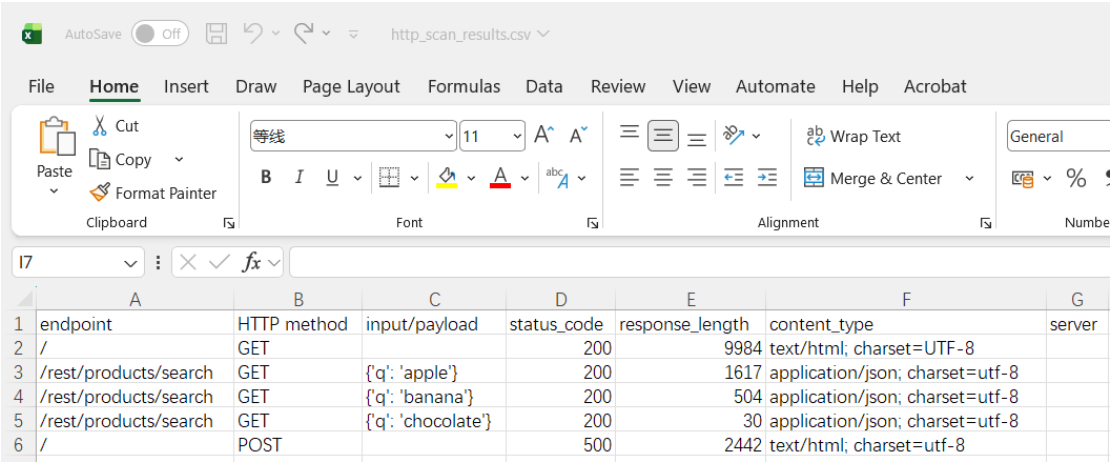
Practice M: Open practice_records.csv and identify its column headings and two rows. Why is CSV useful for the final lab report?

The CSV file contains the column headings endpoint and status, along with two rows of data (/, 200 and /example, 404). CSV is useful for the final lab report because it easily stores structured tabular data that can be conveniently opened, organized, and reviewed in spreadsheet software.

Part D — Your programming assignment (build both scripts yourself)

Task 1 — Develop http_scanner.py

```
PS D:\FSCT\FSCT-8561\Labs\FSCT8561-Labs-Kuangda-Zhu\Lab5> python .\http_scanner.py
Requested URL: http://localhost:3000/
Status: 200
Response Length: 9984
Content-Type: text/html; charset=UTF-8
----------------------------------------
Requested URL: http://localhost:3000/rest/products/search?q=apple
Status: 200
Response Length: 1617
Content-Type: application/json; charset=utf-8
----------------------------------------
Requested URL: http://localhost:3000/rest/products/search?q=banana
Status: 200
Response Length: 504
Content-Type: application/json; charset=utf-8
----------------------------------------
Requested URL: http://localhost:3000/rest/products/search?q=chocolate
Status: 200
Response Length: 30
Content-Type: application/json; charset=utf-8
----------------------------------------
Requested URL: http://localhost:3000/rest/products/search
Status: 500
Response Length: 2442
Content-Type: text/html; charset=utf-8
----------------------------------------
Saved http_scan_results.csv
PS D:\FSCT\FSCT-8561\Labs\FSCT8561-Labs-Kuangda-Zhu\Lab5>
```



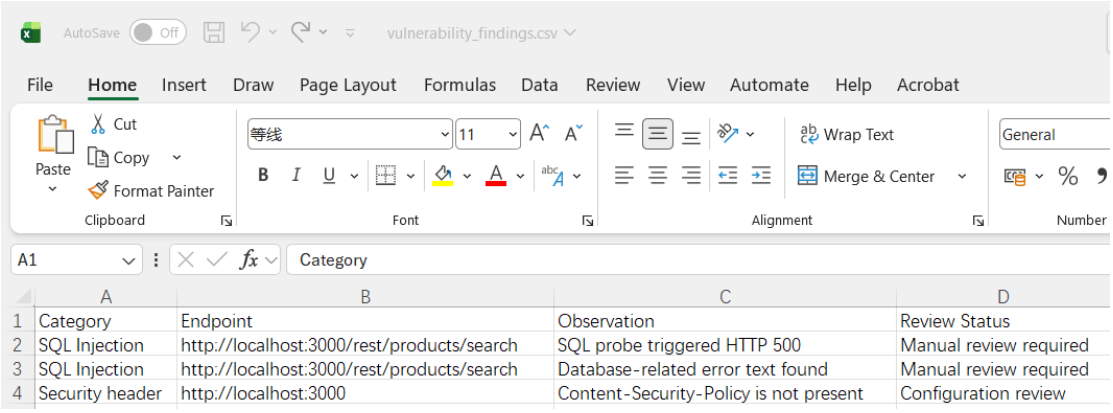
	A	B	C	D	E	F	G
1	endpoint	HTTP method	input/payload	status_code	response_length	content_type	server
2	/	GET		200	9984	text/html; charset=UTF-8	
3	/rest/products/search	GET	{'q': 'apple'}	200	1617	application/json; charset=utf-8	
4	/rest/products/search	GET	{'q': 'banana'}	200	504	application/json; charset=utf-8	
5	/rest/products/search	GET	{'q': 'chocolate'}	200	30	application/json; charset=utf-8	
6	/	POST		500	2442	text/html; charset=utf-8	

As shown in the execution output, the http_scanner.py script ran successfully, sending both GET and POST requests to the target endpoints. All required details—such as

endpoints, methods, input payloads, status codes, response lengths, and content types—were accurately collected and exported to http_scan_results.csv. Since the server did not return a Server header, the server column remains blank (None).

Task 2 — Develop vulnerability_detector.py

```
PS D:\FSCT\FSCT-8561\Labs\FSCT8561-Labs-Kuangda-Zhu\Lab5> python .\vulnerability_detector.py
Category: SQL Injection
Endpoint: http://localhost:3000/rest/products/search
Observation: SQL probe triggered HTTP 500
Review Status: Manual review required
--------------------------------------------------------------------------------
Category: SQL Injection
Endpoint: http://localhost:3000/rest/products/search
Observation: Database-related error text found
Review Status: Manual review required
--------------------------------------------------------------------------------
Category: Security header
Endpoint: http://localhost:3000
Observation: Content-Security-Policy is not present
Review Status: Configuration review
--------------------------------------------------------------------------------
Saved vulnerability_findings.csv
PS D:\FSCT\FSCT-8561\Labs\FSCT8561-Labs-Kuangda-Zhu\Lab5>
```



	A	B	C	D
1	Category	Endpoint	Observation	Review Status
2	SQL Injection	http://localhost:3000/rest/products/search	SQL probe triggered HTTP 500	Manual review required
3	SQL Injection	http://localhost:3000/rest/products/search	Database-related error text found	Manual review required
4	Security header	http://localhost:3000	Content-Security-Policy is not present	Configuration review

As shown in the execution output, the vulnerability_detector.py script ran successfully, detecting and reporting security indicators from the target endpoints. All required details—such as categories, endpoints, observations, and review statuses—were accurately collected and exported to vulnerability_findings.csv.

Task 3 — Server and configuration analysis

- **Which response headers were present and missing, and why the context matters.**

  X-Content-Type-Options was present, while Content-Security-Policy was missing. Context matters because the actual risk level depends on the application's environment (e.g., production vs. local testing) and its data sensitivity.

- **Whether the server reveals potentially useful implementation information.**

  No, the server did not reveal useful implementation details because the Server header was absent (None) in the scan results.

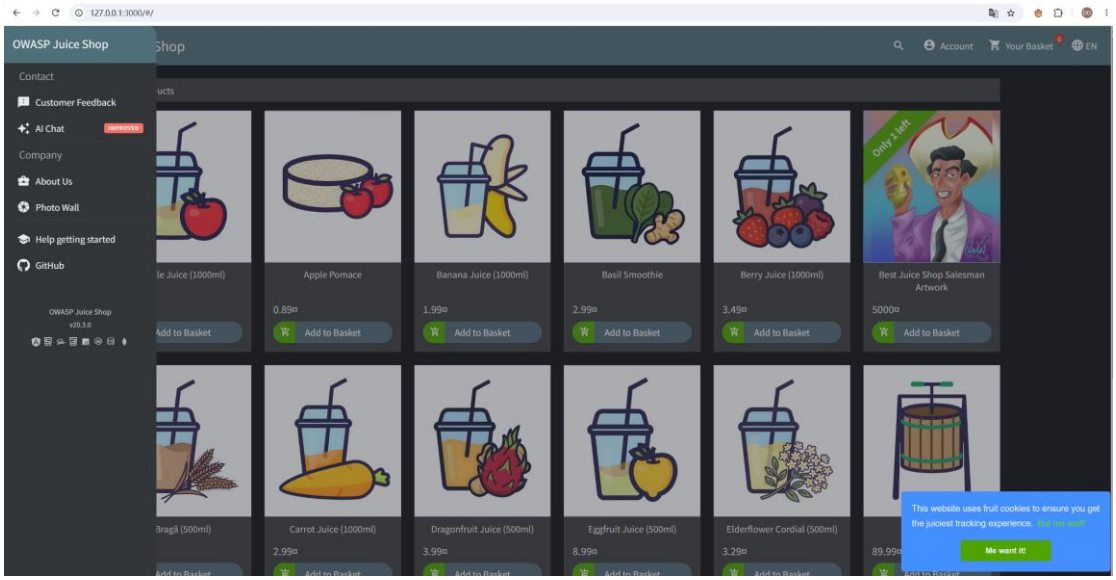
- **Why HTTPS/TLS cannot be meaningfully assessed solely by checking the local HTTP endpoint; describe what you would examine in an actual HTTPS deployment.**

  Local HTTP endpoints use plaintext and cannot evaluate transport security. In a real HTTPS deployment, you must examine SSL/TLS certificate validity, cipher suites, protocol versions, and HSTS implementation.

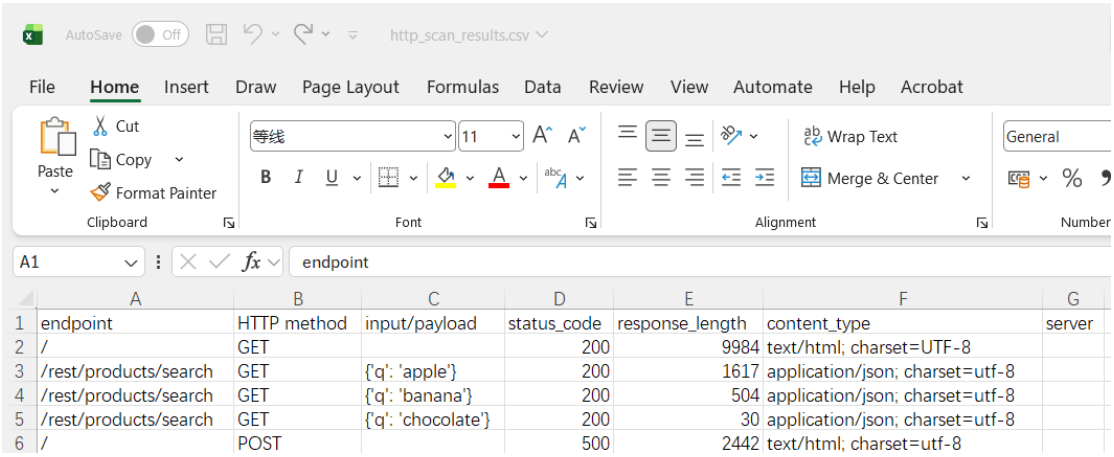
- **Why a missing header or server error does not automatically mean a vulnerability is exploitable.**

  A missing header or a server error (like HTTP 500) indicates a defense gap or input exception, serving only as a security indicator for manual review rather than proof of a confirmed vulnerability.

Task 4 — Evidence and reproducibility

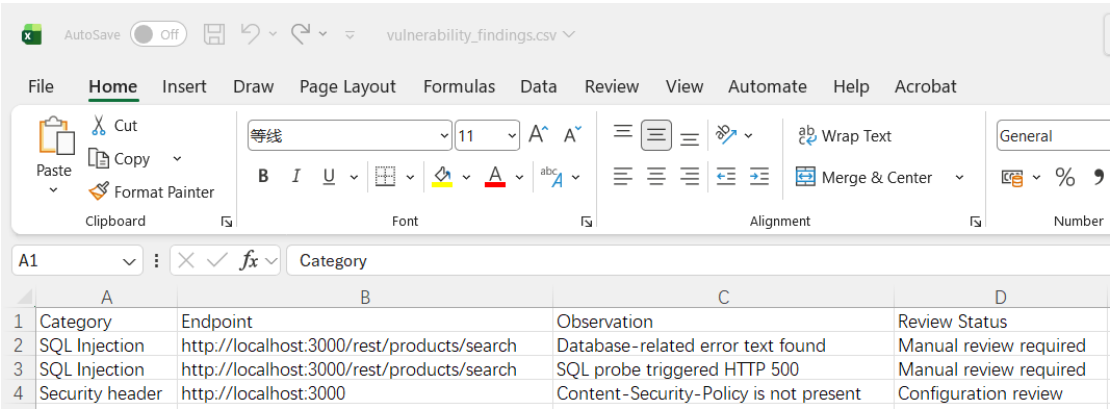


```
PS D:\FSCT\FSCT-8561\Labs\FSCT8561-Labs-Kuangda-Zhu\Lab5> python .\http_scanner.py
Requested URL: http://localhost:3000/
Status: 200
Response Length: 9984
Content-Type: text/html; charset=UTF-8
----------------------------------------
Requested URL: http://localhost:3000/rest/products/search?q=apple
Status: 200
Response Length: 1617
Content-Type: application/json; charset=utf-8
----------------------------------------
Requested URL: http://localhost:3000/rest/products/search?q=banana
Status: 200
Response Length: 504
Content-Type: application/json; charset=utf-8
----------------------------------------
Requested URL: http://localhost:3000/rest/products/search?q=chocolate
Status: 200
Response Length: 30
Content-Type: application/json; charset=utf-8
----------------------------------------
Requested URL: http://localhost:3000/rest/products/search
Status: 500
Response Length: 2442
Content-Type: text/html; charset=utf-8
----------------------------------------
Saved http_scan_results.csv
```



	A	B	C	D	E	F	G
1	endpoint	HTTP method	input/payload	status_code	response_length	content_type	server
2	/	GET		200	9984	text/html; charset=UTF-8	
3	/rest/products/search	GET	{'q': 'apple'}	200	1617	application/json; charset=utf-8	
4	/rest/products/search	GET	{'q': 'banana'}	200	504	application/json; charset=utf-8	
5	/rest/products/search	GET	{'q': 'chocolate'}	200	30	application/json; charset=utf-8	
6	/	POST		500	2442	text/html; charset=utf-8	

```
PS D:\FSCT\FSCT-8561\Labs\FSCT8561-Labs-Kuangda-Zhu\Lab5> python .\vulnerability_detector.py
Category: SQL Injection
Endpoint: http://localhost:3000/rest/products/search
Observation: Database-related error text found
Review Status: Manual review required
--------------------------------------------------------------------------------
Category: SQL Injection
Endpoint: http://localhost:3000/rest/products/search
Observation: SQL probe triggered HTTP 500
Review Status: Manual review required
--------------------------------------------------------------------------------
Category: Security header
Endpoint: http://localhost:3000
Observation: Content-Security-Policy is not present
Review Status: Configuration review
--------------------------------------------------------------------------------
Saved vulnerability_findings.csv
PS D:\FSCT\FSCT-8561\Labs\FSCT8561-Labs-Kuangda-Zhu\Lab5>
```



	A	B	C	D
1	Category	Endpoint	Observation	Review Status
2	SQL Injection	http://localhost:3000/rest/products/search	Database-related error text found	Manual review required
3	SQL Injection	http://localhost:3000/rest/products/search	SQL probe triggered HTTP 500	Manual review required
4	Security header	http://localhost:3000	Content-Security-Policy is not present	Configuration review

Task 5 — Reflection and security analysis

During this comprehensive lab session, our HTTP scanner successfully collected objective evidence such as HTTP status codes, response lengths, and content types, while the vulnerability detector flagged specific security indicators including database error texts, HTTP 500 server errors caused by SQL probes, and a missing Content-Security-Policy header. These outcomes represent factual empirical observations rather than definitive proof of a confirmed exploit. If I were acting as the security lead for the Juice Shop application, I would immediately prioritize investigating and fixing the potential SQL injection vulnerability found on the product search endpoint because it presents an extreme risk of unauthorized data exposure and remote database manipulation, followed closely by implementing the missing security headers to strengthen overall defense-in-depth. Compared to the passive network packet analysis performed in Lab 4, this assignment transitions smoothly into active application-layer testing by allowing us to programmatically send crafted inputs, handle varying payloads, and closely evaluate how the server processes abnormal requests. Furthermore, if the target application utilized secure HTTPS connections instead of plaintext HTTP, a passive network capture would only reveal encrypted ciphertext

during the SSL handshake and data transmission phase, whereas our local Python client can directly inspect the unencrypted response body and headers. Ultimately, automated scanning tools are only helpful as a preliminary step because they rely heavily on basic pattern matching and heuristics, which often lead to false positives that require careful manual verification and human analysis before any finding can be reliably reported as a true, exploitable vulnerability. To ensure robust application security in professional environments, combining automated detection scripts with rigorous manual code review remains an essential industry best practice. By integrating these automated checks into the standard software development lifecycle, development teams can effectively minimize critical blind spots long before production deployment.

Github

https://github.com/bnwzc/FSCT8561-Labs-Kuangda-Zhu